FIRST_VALUE() in SQL

`FIRST_VALUE()` is a window function that returns the value from the first row in a window (partition), based on the ordering you specify.

It is useful when you want to display the first or highest/lowest value alongside every row, without collapsing rows.

1. Basic syntax

```
FIRST_VALUE(column_name) OVER (
   PARTITION BY column_name
   ORDER BY column_name
)
```

- `PARTITION BY` – divides rows into groups.
- `ORDER BY` – determines which row is first.
- `FIRST_VALUE()` – returns the selected column's value from that first row.

2. Example

Consider an `employees` table:

employee	department	salary
Amit	IT	50000
Rahul	IT	70000
Priya	IT	60000
Neha	HR	40000

Ravi	HR	50000

Query: Get the highest salary in each department and display it alongside every employee.

```
SELECT
   employee,
   department,
   salary,
   FIRST_VALUE(salary) OVER (
      PARTITION BY department
      ORDER BY salary DESC
   ) AS highest_salary
FROM employees;
```

Result:

employee	**department**	**salary**	**highest_salary**
Amit	IT	50000	70000
Rahul	IT	70000	70000
Priya	IT	60000	70000
Neha	HR	40000	50000
Ravi	HR	50000	50000

Notice that:

- IT's first row after sorting by salary descending is Rahul (70000).
- HR's first row is Ravi (50000).
- The first value is repeated for every row in that department.

3. FIRST_VALUE() vs ROW_NUMBER() vs MAX()

Function	Purpose
`FIRST_VALUE()`	Returns a column value from the first ordered row
`ROW_NUMBER()`	Assigns a unique sequential number to each row
`MAX()`	Returns the maximum value in a group/window

For example:

```
SELECT
  employee,
  department,
  salary,
  FIRST_VALUE(employee) OVER (
    PARTITION BY department
    ORDER BY salary DESC
  ) AS highest_paid_employee,
  MAX(salary) OVER (
    PARTITION BY department
  ) AS max_salary
FROM employees;
```

Both functions preserve all employee rows, but `FIRST_VALUE()` can return a value from a different column (such as the employee name).

4. Important: FIRST_VALUE() and window frames

You can explicitly define the window frame:

```
FIRST_VALUE(salary) OVER (
  PARTITION BY department
  ORDER BY salary DESC
  ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
```

)

Here:

- `UNBOUNDED PRECEDING` means start from the first row.
- `CURRENT ROW` means end at the current row.

Because the first row remains included, `FIRST_VALUE()` continues returning the first row's salary.

5. Practical use cases

1. Find the highest-paid employee in each department.
2. Find the earliest transaction date for each customer.
3. Display the first order amount for every customer.
4. Compare each employee's salary with the department's highest salary.
5. Display the first event or status in a customer's activity history.

6. FIRST_VALUE() vs LAST_VALUE()

These are complementary functions:

Function	Returns
`FIRST_VALUE()`	Value from the first ordered row
`LAST_VALUE()`	Value from the last row in the window frame

Important Oracle detail: `LAST_VALUE()` can be surprising because its default window frame often ends at the current row. To get the actual last value in the entire department, explicitly specify:

```
LAST_VALUE(salary) OVER (
   PARTITION BY department
   ORDER BY salary DESC
   ROWS BETWEEN UNBOUNDED PRECEDING
        AND UNBOUNDED FOLLOWING
```

)

Remember the distinction: `FIRST_VALUE()` does not necessarily mean the smallest or largest value. It returns the value from whichever row your `ORDER BY` places first.